

# **log/slog**

what is it? why should I care?

# Why now?

3 years in the making 😊💧

*"pfiaux opened on Nov 21, 2023"*

# **What we'll cover**

1. Basic Use
2. Capabilities/design
3. Case studies and ideas

**What we won't cover: history, origin, ...**

[go.dev/blog/slog](https://go.dev/blog/slog)

# You can vibe use it

```
Claude Code v2.1.49

Welcome back Patrick!



Sonnet 4.6 · Claude Pro
~/Code/slog-test

Tips for getting started
Run /init to create a CLAUDE.md file with instructions for Claude

Recent activity
No recent activity

> Initialize a go project that logs "BarnerGo is awesome" using slog.
```

## The code

```
package main

import "log/slog"

func main() {
    slog.Info("BarnerGo is awesome")
}
```

## The output

```
2026/03/11 16:48:34 INFO BarnerGo is
awesome
```

## The code (with data)

```
package main

import "log/slog"

func main() {
    slog.Info("BarnerGo is awesome",
        "bias", "high")
}
```

## The output

```
2026/03/11 16:57:51 INFO BarnerGo is
awesome bias=high
```



# Structured logs

## log vs slog

```
slog.Info("BarnerGo is awesome", "bias", "high")
// Result
// 2026/03/11 17:05:13 INFO BarnerGo is awesome bias=high
log.Printf("BarnerGo is awesome, author bias is %s", "high")
// Result
// 2026/03/11 17:05:13 BarnerGo is awesome, author bias is high
```



# logfmt

Even without an official spec,  
there's plenty to read about *logfmt*,  
i.e. [brandur.org/logfmt](https://brandur.org/logfmt)

*(make observability people less grumpy)*

**what about logrus?**

([github.com/sirupsen/logrus](https://github.com/sirupsen/logrus))

**logrus is *maintained* (for now)**

```
INFO[0000] BarnerGo is awesome
```

```
bias=high
```

# logrus has 2 dependencies

```
require (  
    github.com/sirupsen/logrus v1.9.4 // indirect  
    golang.org/x/sys v0.13.0 // indirect  
)
```

Note: 0 dependencies = less renovate/dependabot PRs

# **log/slog** under the hood

[pkg.go.dev/log/slog](https://pkg.go.dev/log/slog)

Each Logger is associated with a Handler. A Logger output method creates a Record from the method arguments and passes it to the Handler, which decides how to handle it.

There is a default `Logger` accessible through top-level functions (such as `Info` and `Error`) that call the corresponding `Logger` methods.



   **Example handler:** `charmbracelet/log`



# main.go

```
package main

import (
    "log/slog"
    "os"

    "github.com/charmbracelet/log"
)

func setDefaultLogger() {
    handler := log.New(os.Stderr)
    handler.SetReportTimestamp(false)
    logger := slog.New(handler)
    slog.SetDefault(logger)
}

func main() {
    setDefaultLogger()
    importantStuff()
}
```

# important.go

```
package main

// notice no charmbracelet/log here:
import "log/slog"

// note we can also explicitly take a logger
func importantStuff() {
    slog.Info("BarnerGo is awesome", "bias", "high")
}
```

## More abstract logging

- Not imported in every file
- Easily swapped, configured or refactored

```
> go run ./
INFO BarnerGo is awesome bias=high
```

## **Make it easier to switch again later**

You can use it to write all your logging with the standard library

You can later choose how you handle it at bootstrap (or run time) without touching the rest of your code.

**Some practical ideas**

# **Redirecting logs during testing**

(a partial solution)

```
func TestSomethingThatLogs(t *testing.T) {  
    // Redirect logs to t.log so they only show on test failure  
    slog.SetDefault(slog.New(mock.NewTestHandler(t, slog.LevelDebug)))  
  
    //...  
    t.Pass() // no logs  
    // on t.Fail() the logs would print  
}
```



## mocks/logs.go 1/3

```
package mocks

import (
    "context"
    "fmt"
    "log/slog"
    "strings"
    "testing"
)

// NewTestHandler allows us to change the default slog logger during tests
// to use t.Log so that logs are only printed for the failing test cases.
//
// // Usage in a test:
// slog.SetDefault(slog.New(NewTestHandler(t, slog.LevelDebug)))
func NewTestHandler(t *testing.T, level slog.Level) *TestHandler {
    return &TestHandler{
        t:      t,
        level: level,
    }
}

// TestHandler is a slog.Handler that logs to the testing.T logger.
type TestHandler struct {
    t      *testing.T
    level slog.Level
}
```



## mocks/logs.go 2/3

```
// Handle maps slog levels between `t.Logf` and `t.Fatalf` (`slog.LevelError` or higher call `t.Fatalf`)
func (h *TestHandler) Handle(_ context.Context, r slog.Record) error {
    // flatten the args if any
    attributesFlattener := strings.Builder{}
    var err error
    r.Attrs(func(attr slog.Attr) bool {
        _, err = attributesFlattener.WriteString(fmt.Sprintf(" %s=%s", attr.Key, attr.Value))
        continueIfNoError := err == nil
        return continueIfNoError
    })
    if err != nil {
        return err
    }

    // for fatal and panic
    if r.Level > slog.LevelError {
        h.t.Fatalf("%s%s", r.Message, attributesFlattener.String())
        return nil
    }

    h.t.Logf("%s%s", r.Message, attributesFlattener.String())
    return nil
}
```

## mocks/logs.go 3/3

```
// Enabled skips logs below the enabled log level for slog wrapper.  
func (h *TestHandler) Enabled(_ context.Context, level slog.Level) bool {  
    return level >= h.level  
}  
  
// WithAttrs is a noop (unused)  
func (h *TestHandler) WithAttrs(_ []slog.Attr) slog.Handler {  
    return h  
}  
  
// WithGroup is a noop (unused)  
func (h *TestHandler) WithGroup(_ string) slog.Handler {  
    return h  
}
```

Calling

`slog.SetDefault(slog.New(mock.NewTestHandler(t, slog.LevelDebug)))` in each test is ugly.

Perhaps `func TestMain(m *testing.M)` can work, I didn't get around to trying.

## Customizing logs

If a type implements the `LogValuer` interface, the `Value` returned from its `LogValue` method is used for logging.

... For example, you can redact secret information like passwords

## **A few more examples thanks @breml**

- limit size of logged values
- extract slog\_attributes from context

## Limit size of logged values

```
// ...
replaceAttrFunc = logValueMaxSize(MaximumValueLength)

slogHandler = slog.NewTextHandler(
    writer,
    &slog.HandlerOptions{
        Level: level,
        AddSource: debug,
        ReplaceAttr: replaceAttrFunc,
    },
)

// logValueMaxSize is a slog.ReplaceAttr function, which limits the size
// of log values to the given limit.
func logValueMaxSize(limit int) func(groups []string, attr slog.Attr) slog.Attr {
    return func(groups []string, attr slog.Attr) slog.Attr {
        if attr.Key == slog.LevelKey || attr.Key == slog.SourceKey {
            return attr
        }

        switch attr.Value.Kind() {
        case slog.KindAny, slog.KindString:
            if attr.Key == slog.MessageKey {
                break
            }

            val := attr.Value.String()
            if len(val) > limit {
                val = val[:limit] + "... (truncated)"
            }

            attr.Value = slog.StringValue(val)
        }

        return attr
    }
}
```

(<https://github.com/FuturFusion/operations-center>)

# Conclusions



# It's a tool

- Find out what it's useful for
- Find out **when** it's useful
  - Only new code?
  - Active code?
  - Rewrite your entire legacy code base to use it?



## It's in the standard lib

*"A little copying is better than a little dependency."*

- Bumping libraries is work (even if automated)
- **Bumping libraries doesn't update deployments**

# Thanks

Special thanks to Open Systems for kindly letting me use some of the code samples shown here

Thanks to you go community